

## Synchronization in java

Synchronization is the process of coordinating access to shared resources in a multi-threaded environment. When multiple threads run simultaneously, they might try to access and modify the same resource (like a variable or an object). If they're not managed properly, this can lead to inconsistent data, known as race conditions.

### Why Do We Need Synchronization?

Imagine two threads trying to update the same bank account balance at the same time. If both threads read the balance before either writes back the new value, one thread's update may overwrite the other's, causing the final balance to be incorrect. Synchronization helps to ensure that only one thread can access a resource at a time, preventing these types of issues.

### How Does Synchronization Work in Java?

In Java, synchronization can be achieved using the `synchronized` keyword, which can be applied to methods or blocks of code. Here's how it works:

- 1. Synchronized Methods:** When a method is declared with the `synchronized` keyword, a thread must acquire the lock for the object (or class, if it's a static method) before it can enter that method.

```
public synchronized void withdraw(int amount) {  
    if (balance >= amount) {  
        balance -= amount;  
    }  
}
```

- 2. Synchronized Blocks:** You can also synchronize specific sections of code that access shared resources rather than the entire method. This is useful for improving performance by reducing the scope of the lock.

```
public void withdraw(int amount) {  
    synchronized (this) {  
        if (balance >= amount) {  
            balance -= amount;  
        }  
    }  
}
```

- 3. Class Level Synchronization:** If you declare a static method as synchronized, it locks the class itself, preventing any other thread from accessing any synchronized static method of that class.

```
public static synchronized void staticMethod() {  
    // some code  
}
```

**Note**

- **Locks:** When a thread acquires a lock, other threads trying to enter the synchronized method or block will wait until the lock is released.
- **Avoid Deadlocks:** Be cautious of situations where two threads are waiting on each other to release locks, causing them to be stuck indefinitely. Always try to use a consistent locking order.
- **Performance:** Synchronization can lead to a performance hit if overused. Only use it where necessary.

**Visualization of Bank Account Example**

- 1. Initial State** - Imagine we start with a bank account that has a balance of 0.

```
+-----+  
|  Bank Account  |  
|-----|  
| Balance: 0     |  
+-----+
```

- 2. Create Threads**

**We have two threads:**

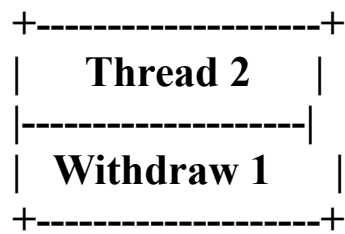
**Thread 1:** This thread is responsible for depositing money.

**Thread 2:** This thread is responsible for withdrawing money.

**Thread 1 (Deposit):**

```
+-----+  
|  Thread 1     |  
|-----|  
| Deposit : 1   |  
+-----+
```

### **Thread 2 (Withdraw):**



### **3. Thread Execution Timeline**

#### **Without Synchronization (Race Condition Example)**

- 1. Thread 1 reads balance of 0.**
- 2. Thread 2 also reads balance of 0 (before Thread 1 updates).**
- 3. Thread 1 writes back balance as 1.**
- 4. Thread 2 writes back balance as 0, as it read the initial balance before Thread 1 updated it.**

**Thread 1: Read Balance: 0**

**Thread 2: Read Balance: 0**

|--- Deposit --> Balance: 1

|                    |

|                    |--- Withdraw --> Balance: 0

|

**Final Balance: 0 (Incorrect)**

#### **4. With Synchronization**

- 1. Thread 1 acquires the lock to perform the deposit.**
- 2. Thread 1 updates balance to 1.**
- 3. Thread 1 releases the lock.**
- 4. Thread 2 acquires the lock to perform the withdrawal.**
- 5. Thread 2 reads balance as 1 and updates to 0.**
- 6. Thread 2 releases the lock.**

**Final Balance With Synchronization: 0 (correct!)**

**Thread 1: Lock Acquired**

- Read Balance: 0

- Deposit: 1

- Write Balance: 1

**Thread 1: Releases Lock**

**Thread 2: Lock Acquired**

- Read Balance: 1

- Withdraw: 1  
- Write Balance: 0  
Thread 2: Releases Lock

Final Balance: 0 (Correct)

#### Summary

- **Initial State:** The bank account starts with a balance of 0.
- **Race Condition Scenario:** Both threads read the balance simultaneously, causing one thread's change to be overwritten by another, resulting in an incorrect final balance.
- **Synchronized Scenario:** The synchronization mechanism ensures that one thread completes its operation (deposit or withdrawal) before the other thread can access or modify the balance, resulting in the correct account balance.